

Room et l'offline-first : la base locale comme source de vérité

Entity, DAO, Database — et pourquoi le hors-ligne d'abord
Mini-TP « Trois requêtes »



Offline-first : la base d'abord, le réseau ensuite

■ Modèle naïf : le réseau d'abord

- L'écran demande au réseau et affiche la réponse
- Pas de réseau → pas d'application
- La moindre coupure devient une panne
- Le modèle par défaut des applications conçues en couverture continue

■ Offline-first : la base d'abord

- L'écran lit TOUJOURS la base locale
- Le réseau alimente la base quand il est là
- L'application fonctionne toujours
- La coupure n'est plus une panne : c'est un retard de synchronisation

« La base locale n'est pas un cache : c'est la source de vérité. Le réseau ne fait que la nourrir. »

Le contexte malgache rend ce choix évident : connectivité inégale, coupures fréquentes, coût des données.



Room en trois éléments — et c'est tout

1 @Entity — une table

Une data class = une table.

Chaque propriété = une colonne.

@PrimaryKey = la clé.

Un Double? devient une colonne NULL autorisée.

2 @Dao — les requêtes

Une interface : chaque fonction porte son SQL en annotation.

Le SQL est VÉRIFIÉ À LA COMPILATION — une faute de colonne ne compile pas.

3 @Database — l'assemblage

Liste les entités, expose les DAO.

Le builder crée le fichier de base sur le téléphone.

Le compilateur génère tout le reste.

Le SQL que vous connaissez déjà — simplement posé dans des annotations, et vérifié par le compilateur.



Le DAO : Flow ou suspend — un choix qui a du sens

ProduitDao — les fonctions fournies du projet

```
@Dao
interface ProduitDao {

    // Un ROBINET : ré-émet à chaque changement
    @Query("SELECT * FROM produits ORDER BY nom ASC")
    fun tousLesProduits(): Flow<List<Produit>>

    // Une question PONCTUELLE : suspend
    @Query("SELECT * FROM produits WHERE id = :id")
    suspend fun parId(id: Int): Produit?

    @Insert
    suspend fun insererTous(p: List<Produit>)

}
```

Le critère de choix

- « Je veux être tenu au courant » → Flow : l'écran se met à jour tout seul
- « Je pose une question ponctuelle » → suspend
- suspend, parce qu'une lecture disque ne doit pas bloquer l'interface (séance 2)
- Room refuse de compiler une requête bloquante sur le thread principal

Et pour ÉCRIRE dans la base ? Quatre annotations, toutes suspend

@Insert

ajoute une ligne

@Update

modifie — par la clé primaire de l'objet

@Delete

supprime — par la clé primaire de l'objet

@Query("DELETE ...")

agir sur un CRITÈRE, pas sur un objet

Le Flow de la séance 2 prend enfin tout son sens : la base émet, le ViewModel transforme, l'écran se recompose.



Démonstration : la coupure n'est pas une panne

1 Réseau activé

La collecte apparaît immédiatement, puis passe en « synchronisée ». Le serveur la reçoit.

→ L'utilisateur n'a attendu que la base : zéro seconde de réseau.

2 Réseau coupé

Deux ou trois collectes enregistrées : elles s'affichent aussitôt, marquées « en attente ».

→ L'application fonctionne entièrement. Rien n'est bloqué.

3 La preuve

Fermer l'application, la rouvrir : les collectes sont toujours là.

→ Les données sont ACQUISES, en base. Elles ne peuvent plus être perdues.

4 Le réseau revient

« Synchroniser » : la file se vide, les marques disparaissent, le serveur se remplit.

→ La coupure n'était pas une panne : c'était un retard.

Le réseau est simulé par un interrupteur dans l'application (faux serveur en mémoire) : la démonstration reste reproductible, sans dépendre de la connexion de la salle.



Les lignes qui font toute la différence

Le geste central : la base d'abord

```
fun enregistrerCollecte(...) {  
    viewModelScope.launch {  
        val c = Collecte(...,  
            synchronisee = false)  
  
        dao.inserer(c)      // 1. LA BASE  
        synchroniser()     // 2. le réseau  
    }  
}
```

Inverser ces deux lignes — envoyer d'abord, écrire ensuite — donnerait le modèle naïf : celui qui perd la donnée quand le réseau manque.

La boucle de synchronisation

```
for (c in dao.enAttente()) {  
    val recue = serveur.envoyer(c)  
    if (recue)  
        dao.modifier(  
            c.copy(synchronisee = true))  
    else break // on réessaiera  
}
```

Le drapeau synchronisee distingue « acquis localement » de « remonté au serveur ». Un booléen suffit — et copy() de la séance 1 sert encore ici.

Dans une vraie application

Le faux serveur devient une interface Retrofit · le bouton « Synchroniser » devient WorkManager, qui programme la reprise et survit à la fermeture. Le reste — l'ordre base puis-réseau, le drapeau, la boucle — ne change pas.



Les préférences : quand une base serait disproportionnée

Une préférence, c'est...

- le mode d'affichage choisi
- le nom du collecteur connecté
- la date de dernière synchronisation
- le seuil de stock, le thème sombre

Point commun : une valeur unique, sans structure, qu'on relit au démarrage — et qu'on n'INTERROGE jamais.

DataStore — trois idées, et c'est tout

```
// 1. LA CLÉ : un nom et un type
val MODE = stringPreferencesKey("mode_affichage")

// 2. LIRE : un Flow — l'écran suit tout seul
val mode: Flow<String> = datastore.data
    .map { prefs -> prefs[MODE] ?: "nom" }

// 3. ÉCRIRE : suspend — jamais sur le thread UI
suspend fun changerMode(m: String) {
    datastore.edit { prefs -> prefs[MODE] = m }
}
```

	Room	DataStore
Pour quoi	Des enregistrements : produits, collectes	Des réglages : un mode, un nom, une date
On interroge ?	Oui — trier, filtrer, agréger, joindre	Non — on lit une clé, on écrit une clé
Forme	Tables, SQL vérifié à la compilation	Clés typées, aucune structure
Lecture	Flow ou suspend, selon l'usage	Flow — même réflexe

DataStore remplace SharedPreferences, qui écrivait sur le thread de l'interface.

Le critère : on veut interroger → Room • on veut retenir un réglage → DataStore.



Mini-TP 7 · « Trois requêtes »

1 Lire et prédire

Lire Donnees.kt ; prédire : qu'affiche l'écran avant les TODO ? et après fermeture complète puis réouverture, les données sont-elles là ?

2 Les trois requêtes

TODO 1 : tri par prix décroissant, prix nuls en dernier. TODO 2 : filtre par seuil de stock (paramètre). TODO 3 : SUM du stock total → Flow<Double?>.

3 Brancher et observer

Suivre le bloc ÉTAPE 3 du ViewModel : brancher au moins un mode + le stock total. Observer : où se place le litchi dans le tri par prix, et pourquoi ?

4 Voie ouverte — juger un test

Faire générer un test d'une requête par l'IA ; en 3 lignes : que teste-t-il RÉELLEMENT, et que faudrait-il vérifier en plus ?

Vos requêtes rendent des Flow : changez une donnée, l'écran se met à jour tout seul. C'est la boucle complète du module.

